

Projet - Classification d'intentions client pour un système de support automatisé

Contexte

Une entreprise reçoit chaque jour des milliers de messages clients (tickets, tweets, emails). Pour accélérer le traitement, elle souhaite **router automatiquement chaque message vers la bonne catégorie d'intention** (demande de remboursement, problème technique, suivi de commande, etc.). Vous êtes chargé de concevoir, entraîner, comparer et déployer un système de classification d'intentions basé sur le Deep Learning.

Objectifs pédagogiques

Le projet doit couvrir l'ensemble du cours :

- Préparation et nettoyage de données textuelles
- Comparaison d'architectures séquentielles (RNN, LSTM, BiLSTM, Transformer from scratch, Transformer pré-entraîné)
- Application des techniques d'optimisation et de régularisation
- Déploiement du meilleur modèle via une API

Dataset

Option principale recommandée : Bitext Customer Support LLM Chatbot Training Dataset

- ~27 000 paires (message client, intention)
- 27 intentions réparties en 11 catégories (ACCOUNT, ORDER, REFUND, etc.)
- Disponible sur Hugging Face :
`bitext/Bitext-customer-support-llm-chatbot-training-dataset`
- Anglais, propre, équilibré

Option alternative : Customer Support on Twitter (Kaggle)

- ~3 millions de tweets, plus bruité, plus réaliste
- Demande un travail de labellisation ou de filtrage plus important

Option francophone (si préférée) : Amazon Reviews Multi (fr) détourné en classification multi-classes, ou MASSIVE (Amazon) avec ~20 000 énoncés français labellisés en 60 intentions.

L'étudiant doit justifier son choix de dataset.

Travail attendu

1. Préparation des données

- Nettoyage (casse, ponctuation, caractères spéciaux, gestion des emojis si applicable)
- Tokenisation et construction du vocabulaire
- Padding et gestion des longueurs de séquence
- Split train / validation / test stratifié
- Analyse exploratoire (distribution des classes, longueur des messages, mots fréquents)
- Discussion du déséquilibre éventuel des classes et stratégie adoptée

2. Modélisation et comparaison

L'étudiant doit implémenter et comparer **au minimum 4 architectures** :

Modèle	Détails attendus
Baseline RNN simple	Embedding + RNN + tête de classification (MLP)
LSTM ou BiLSTM	Embedding + LSTM/BiLSTM + MLP
Transformer from scratch	Embedding + positional encoding + encodeur Transformer (multi-head attention) + MLP
Transformer pré-entraîné	DistilBERT ou CamemBERT (si dataset français) fine-tuné

Les étudiants sont libres de **combiner les architectures** : par exemple un **encodeur Transformer comme extracteur** suivi d'une **tête MLP custom**, ou un **CNN 1D + LSTM** sur les embeddings. Toute hybridation justifiée est valorisée.

3. Optimisation et régularisation

Le meilleur modèle doit faire l'objet d'un travail approfondi :

- Comparaison d'optimiseurs (SGD, Adam, AdamW)

- Scheduler de learning rate (cosine, ReduceLROnPlateau, warmup)
- Régularisation : dropout, weight decay, early stopping
- Recherche d'hyperparamètres (grid search ou random search documentée)
- Courbes d'apprentissage commentées (détection d'overfitting, etc.)

4. Évaluation

- Accuracy, précision, rappel, F1 (macro et weighted)
- Matrice de confusion analysée (quelles intentions sont confondues, pourquoi)
- Analyse d'erreurs sur quelques exemples concrets
- Mesure de la latence d'inférence par modèle

5. Déploiement

- Export du meilleur modèle (TorchScript ou ONNX)
- API FastAPI avec au minimum un endpoint `/predict` qui prend un texte et renvoie l'intention prédite + score de confiance
- Documentation Swagger fonctionnelle
- Bonus : Dockerfile, endpoint `/health`, gestion de batch, quantization comparée

Livrables

1. Rapport technique (PDF, 10 à 15 pages) structuré ainsi :

- Introduction et choix du dataset
- Préparation des données
- Description des architectures testées
- Protocole expérimental
- Résultats commentés (tableaux comparatifs, courbes)
- Analyse d'erreurs
- Déploiement
- Conclusion et pistes d'amélioration

2. Code source sur dépôt Git (ou archive zip) avec :

- Notebooks d'expérimentation lisibles et commentés
- Script d'entraînement reproductible
- Code de l'API FastAPI
- README expliquant comment lancer chaque partie
- `requirements.txt`

3. **Démonstration de l'API** : capture d'écran ou court enregistrement montrant l'API qui répond à 2-3 requêtes.

Grille d'évaluation

Documentation et explications - 60 %

Critère	Points
Qualité du rapport (clarté, structure, orthographe)	10
Justification du dataset et analyse exploratoire	10
Explication des architectures et choix de conception	15
Analyse des résultats et comparaison argumentée	15
Analyse d'erreurs et recul critique	10

Code et implémentation - 40 %

Critère	Points
Qualité du code (lisibilité, organisation, reproductibilité)	10
Implémentation correcte des 4+ architectures	10
Travail d'optimisation et régularisation effectif	10
API FastAPI fonctionnelle et bien structurée	10

Note

- Commencez petit : faites tourner un RNN de bout en bout (data → modèle → API simpliste) avant de chercher la performance